

AI-driven Learning for Education and Cooperation (Alec)

Documentazione attraverso protocollo CRISP-DM del progetto
Alec.

Davide Viola

21 ottobre 2025

<https://github.com/davidviola63/Alec-AIProject.git>

Indice

1	Business Understanding	3
1.1	Contesto	3
1.2	Obiettivi di business	3
1.3	Requisiti	4
1.3.1	Requisiti funzionali	4
1.3.2	Requisiti non funzionali	4
1.4	Criteri di successo	5
1.5	Ambito del progetto	5
2	Data Understanding	5
2.1	Raccolta dei dati	6
2.2	Descrizione dei dati	6
2.3	Esplorazione dei dati	7
2.4	Verifica della qualità dei dati	7
3	Data Preparation	8
3.1	Selezione dei dati	8
3.2	Pulizia dei dati	9
3.3	Trasformazione dei dati e dataset finale	9
4	Modeling	10
4.1	Introduzione e scelta del modello	10
4.2	Flusso di elaborazione del messaggio	11
4.2.1	1. Ricezione del messaggio	11
4.2.2	2. Recupero del contesto	11
4.2.3	3. Valutazione (Judge)	11
4.2.4	4. Decisione del Response Gate	11
4.2.5	5. Generazione della risposta	12
4.2.6	6. Gestione dei comandi speciali	12
4.2.7	7. Output finale	12

4.3	Progettazione dei prompt	12
4.4	Requisiti funzionali affrontati nella fase di modeling	13
4.5	Scelta dei parametri di modellazione	14
5	Evaluation	15
5.1	Obiettivi della fase di valutazione	15
5.2	Metodologia di valutazione	15
5.3	Valutazione delle funzionalità interattive	16
5.4	Riscontro rispetto ai requisiti funzionali	16
5.5	Valutazione dei requisiti non funzionali	16
5.6	Considerazioni finali sulla fase di valutazione	17
6	Deployment	17
6.1	Introduzione	17
6.2	Caratteristiche del prototipo	17
6.3	Limitazioni e miglioramenti futuri	18
6.4	Valutazione del deployment	18
6.5	Conclusione	19

1 Business Understanding

1.1 Contesto

Il progetto, intitolato *AI-driven Learning for Education and Cooperation* (Alec), nasce per dare concretezza allo studio e all'analisi eseguiti sui LLM in ambito educativo svolto da me. A corredo del progetto sarà disponibile anche il paper relativo allo studio consultabile su GitHub. L'obiettivo di Alec è quello di proporsi come tutor virtuale a supporto della metodologia di peer learning tra studenti, offrendo un aiuto personalizzato e guidato dai documenti didattici revisionati e forniti dai docenti. Così facendo si offre un duplice beneficio: ridurre il carico di lavoro dei docenti e garantire agli studenti uno strumento in grado di offrire risposte di supporto, sempre coerenti con il materiale del corso, in modo continuo e stimolante.

1.2 Obiettivi di business

L'obiettivo principale del progetto **ALEC** è esplorare l'impiego dei modelli LLM in un contesto educativo collaborativo, al fine di migliorare la qualità e l'efficacia dello studio tra pari. Il sistema mira a integrare l'intelligenza artificiale come *peer tutor* non invasivo, capace di stimolare il pensiero critico e supportare i processi cognitivi degli studenti attraverso un'interazione contestuale, naturale e guidata.

In particolare, il progetto persegue i seguenti obiettivi:

- **Ridurre la diffusione di errori concettuali** durante le sessioni di peer-learning, grazie a un sistema che valuta la correttezza dei contributi e interviene in modo mirato in caso di incongruenze o richieste esplicite di chiarimento.
- **Fornire un supporto didattico personalizzato e progressivo** attraverso un meccanismo di *scaffolding* multilivello, che accompagna lo studente nella costruzione autonoma della conoscenza, partendo da suggerimenti generali fino ad arrivare alla spiegazione completa del concetto.
- **Garantire coerenza con il materiale didattico ufficiale** mediante tecnologia RAG (Retrieval-Augmented Generation) che recupera i contenuti rilevanti dai documenti caricati dal docente.
- **Offrire ai docenti e agli studenti strumenti di monitoraggio avanzati** tracciando la partecipazione, la pertinenza e la qualità degli interventi, ottenendo un report che permetta una visione generale su quanto c'è da migliorare nell'attività di studio.
- **Sviluppare una piattaforma modulare e scalabile**, facilmente estendibile ad altri corsi o contesti formativi, mantenendo il controllo completo sui contenuti e sull'aderenza ai programmi didattici.

1.3 Requisiti

1.3.1 Requisiti funzionali

I requisiti funzionali definiscono le capacità operative che il sistema deve possedere per soddisfare gli obiettivi di business. Essi descrivono il comportamento atteso del sistema in termini di interazione, coerenza didattica e tracciabilità delle attività educative.

- **RF-1** Il sistema deve utilizzare i materiali didattici forniti dal docente come base di conoscenza principale, garantendo che le risposte siano coerenti con le fonti accademiche e includano un riferimento esplicito ai contenuti consultati.
- **RF-2** Il sistema deve adattare le proprie risposte in funzione della percentuale di errore rilevata nelle affermazioni dello studente, mantenendo un comportamento coerente, mirato a promuovere la riflessione e il ragionamento critico.
- **RF-3** Il sistema deve guidare lo studente attraverso un percorso di apprendimento progressivo basato su spiegazioni gradualmente articolate (*scaffolding*) in livelli di supporto crescente.
- **RF-4** Il sistema deve gestire conversazioni multi-turno in modalità multi-utente, preservando il contesto del dialogo e distinguendo i diversi interlocutori.
- **RF-5** Il sistema deve monitorare la partecipazione degli utenti, tracciando la frequenza, la pertinenza e la qualità dei loro interventi, permettendo la visione di queste statistiche agli stessi.
- **RF-6** Al termine di ogni sessione, il sistema deve produrre un report sintetico destinato a docenti e studenti, contenente una valutazione complessiva della partecipazione, i punti di forza, le aree di miglioramento e suggerimenti mirati per lo studio individuale.

1.3.2 Requisiti non funzionali

I requisiti non funzionali definiscono le qualità e i vincoli della soluzione web, mantenendo verifiche semplici per l'MVP.

- **RNF-1 (Accuratezza)** Raggiungere un livello di accuratezza della risposta elevato (risposte corrette >90%)
- **RNF-2 (Prestazioni back-end)** Mantenere un tempo medio di risposta del tutor $\leq 40s$ su test di laboratorio.
- **RNF-3 (Trasparenza)** Includere, dove possibile, una citazione alla fonte interna per ogni correzione.
- **RNF-4 (Semplicità)** Semplicità nell'utilizzo del modello per gli utenti finali.
- **RNF-5 (Costo)** Il prototipo deve essere implementato totalmente in maniera gratuita.

1.4 Criteri di successo

Il progetto si considera completato con successo quando il prototipo rispetta i vincoli descritti nei requisiti non funzionali e ha implementato tutti i requisiti funzionali, inoltre deve dimostrare, attraverso test di laboratorio, di poter operare in maniera stabile e comprensibile. In particolare, si richiede che le correzioni fornite siano sufficientemente accurate, che l'interazione avvenga con tempi di risposta adeguati e che l'interfaccia risulti usabile senza necessità di competenze tecniche avanzate.

Il successo non si misura soltanto sugli aspetti tecnici, ma anche sulla capacità del sistema di mostrarsi come uno strumento efficace per l'apprendimento collaborativo: deve infatti risultare uno strumento innovativo e stimolante attraverso le spiegazioni progressive e personalizzate. Infine i dati appresi durante la conversazione dovranno risultare un punto di partenza per migliorare i percorsi didattici per docenti e studenti.

1.5 Ambito del progetto

Il progetto viene sviluppato da un singolo studente come parte di un tirocinio universitario. Il tempo a disposizione è limitato alla durata del tirocinio e le risorse utilizzabili consistono principalmente in un ambiente di sviluppo locale, librerie software open source e l'accesso a modelli LLM tramite API o strumenti equivalenti.

Nella fase iniziale produrremo un Minimum Viable Product con focus su un singolo corso di studio (Fondamenti di Intelligenza Artificiale) e l'attenzione sarà rivolta esclusivamente alla funzionalità di tutoring: rilevazione di affermazioni scorrette o richieste esplicite di aiuto, fornitura di spiegazioni graduali adattate al livello dello studente, tracciamento della partecipazione e generazione di un report finale. Funzionalità aggiuntive come la proposta di esercizi pratici, l'integrazione di link esterni sicuri e la realizzazione di una dashboard avanzata per i docenti saranno prese in considerazione in cicli successivi, una volta validate le basi del prototipo.

2 Data Understanding

Lo scopo di questa fase è acquisire una chiara comprensione dei dati che costituiranno la base di conoscenza del progetto Alec. In particolare, si tratta dei materiali didattici forniti dai docenti in formato PDF, che verranno analizzati per verificarne la struttura, la qualità e l'idoneità all'utilizzo all'interno del sistema.

Il processo seguito si articola in quattro passaggi principali:

1. **Raccolta:** identificazione e acquisizione dei file PDF pertinenti al corso.
2. **Descrizione:** analisi dei formati e delle principali caratteristiche (numero di documenti, pagine, tipologia di contenuti).
3. **Esplorazione:** valutazione della varietà e complessità dei materiali, con attenzione a testi narrativi, elenchi, formule e tabelle.
4. **Verifica della qualità:** individuazione di criticità come incoerenze terminologiche, mancanza di completezza o contenuti difficilmente leggibili.

L'obiettivo finale è produrre un inventario dei materiali disponibili, evidenziandone le caratteristiche principali e i potenziali problemi da affrontare nelle fasi successive di preparazione e modellazione.

2.1 Raccolta dei dati

La raccolta dei dati è avvenuta attraverso la selezione dei materiali didattici ufficiali messi a disposizione dai docenti del corso. Tali materiali, costituiti principalmente da file in formato PDF (dispense, slide e articoli), sono stati acquisiti tramite una cartella condivisa predisposta dal laboratorio di intelligenza artificiale presente sulla piattaforma e-learning dell'università, nel corso di studio dell'anno 2024 predisposto per le matricole "Resto 1" del dipartimento di informatica.

Per garantire coerenza e affidabilità, sono stati inclusi unicamente i documenti revisionati e approvati dai docenti, escludendo eventuali appunti personali o risorse di terze parti non validate. I file sono stati organizzati in sottocartelle per modulo e argomento, con una convenzione di nomenclatura uniforme (`Corso/Modulo/Titolo.pdf`) ed è possibile ispezionarli nella cartella `pdf_corsi`, al fine di agevolarne la consultazione e la successiva fase di preparazione. Ad esempio è possibile trovare il file PDF riguardante gli algoritmi di ricerca locale non informata nel percorso:

```
Data_Extraction_&Preparation/pdf_corsi/FIA/Algoritmi_ricerca/
algoritmi_di_ricerca_non_informata.pdf
```

2.2 Descrizione dei dati

La seguente tabella sintetizza le caratteristiche principali dei materiali raccolti. Viene presentata come scheda descrittiva, utile a fornire una panoramica generale del dataset disponibile.

Caratteristica	Descrizione
Numero di documenti	26
Dimensione totale approssimata	148 Mb
Tipologia dei documenti	Prevalentemente dispense e slide, seguono estratti di libro, alcuni riferimenti di esercizi e appunti
Lingua	Italiano, Inglese
Formato prevalente	PDF testuale
Elementi presenti	tabelle, formule matematiche, immagini, figure, elenchi puntati
Struttura interna	titoli e sezioni in tutti mentre sottosezioni e numerazione solo negli estratti del libro

Tabella 1: Scheda di descrizione dei dati raccolti

2.3 Esplorazione dei dati

L'esplorazione dei dati ha lo scopo di analizzare il contenuto dei materiali raccolti per comprendere meglio la varietà, la complessità e le caratteristiche rilevanti. Dall'analisi preliminare emergono i seguenti aspetti:

- **Tematiche trattate:** i documenti coprono i contenuti fondamentali del corso di *Fondamenti di Intelligenza Artificiale*. In particolare sono presenti:
 - Algoritmi di ricerca (strategie informate, non informate e locale).
 - Algoritmi con avversari (minimax e varianti).
 - Tecniche di machine learning, tra cui clustering, classificazione e regressione, con accenni ai foundation models e large language models.
- **Tipologia dei contenuti:** le slide e le dispense includono testo descrittivo ed elenchi puntati, frequenti immagini esplicative accompagnate da didascalie, schemi concettuali, oltre a scarse formule matematiche ed esempi di applicazione. Si può riscontrare anche la presenza di pseudocodice nelle slide sugli algoritmi di ricerca.
- **Livello del linguaggio:** il linguaggio adottato è formale e preciso, ma al tempo stesso scorrevole e non eccessivamente rigoroso. Questo lo rende accessibile e comprensibile anche a studenti che affrontano per la prima volta gli argomenti trattati.
- **Eterogeneità dei documenti:** l'eterogeneità risulta complessivamente molto sottile. I materiali si adattano bene alla tematica trattata ed evitano ripetizioni eccessive, mantenendo uno stile uniforme tra i diversi PDF. La lunghezza media dei documenti è di circa 45 pagine, valore che li rende gestibili e coerenti per lo studio.

2.4 Verifica della qualità dei dati

La verifica della qualità dei dati si occupa di valutare l'affidabilità e l'idoneità dei materiali raccolti. Dall'analisi preliminare emergono le seguenti considerazioni:

- **Leggibilità:** i PDF sono in gran parte testuali e quindi direttamente processabili. Non sono state individuate scansioni di bassa qualità che possano compromettere l'estrazione dei contenuti.
- **Uniformità:** lo stile dei documenti è omogeneo e facilita la coerenza complessiva del dataset. L'unica eccezione è rappresentata dal materiale per esercitazioni, che presenta uno stile leggermente diverso.
- **Completezza:** i contenuti raccolti risultano coerenti con il programma del corso, coprendo sia gli argomenti di base (algoritmi di ricerca e con avversari) sia quelli avanzati (machine learning, LLM, fasi CRISP-DM).
- **Ridondanza:** non sono emerse ripetizioni eccessive o ridondanze tra i documenti, favorendo una gestione snella del corpus.

- **Potenziati criticità:** la presenza di formule matematiche, schemi e alcune tabelle potrebbe richiedere attenzione nella fase di preparazione, in quanto non sempre sono rappresentate in formato testuale.
- **Dimensioni:** la lunghezza media dei documenti, pari a circa 45 pagine, si dimostra gestibile e adeguata per le successive fasi di segmentazione e indicizzazione.

3 Data Preparation

La fase di *Data Preparation* ha l'obiettivo di trasformare i materiali raccolti in un dataset strutturato, pulito e pronto per essere utilizzato dal sistema. Mentre la *Data Understanding* si è concentrata sull'analisi preliminare delle fonti, in questa fase vengono eseguite tutte le operazioni necessarie per rendere i contenuti effettivamente utilizzabili. Le attività comprendono la selezione dei documenti pertinenti, la pulizia e normalizzazione dei file, la loro segmentazione in unità più piccole e l'arricchimento con metadati. Il risultato finale è un corpus coerente e ben organizzato, in grado di supportare l'interazione tra studenti e tutor virtuale, garantendo risposte basate esclusivamente sui materiali didattici forniti dai docenti.

3.1 Selezione dei dati

A partire dall'insieme di documenti individuati nella fase di *Data Understanding*, è stata effettuata una selezione mirata a definire il corpus di riferimento. Questa operazione non ha comportato l'acquisizione di nuovi materiali, ma piuttosto la riorganizzazione di quelli già disponibili. In particolare:

- Sono stati confermati i materiali ufficiali forniti dai docenti come base di conoscenza principale.
- È stata distinta la documentazione teorica (dispense, slide, articoli) dai materiali di esercitazione, mantenendo entrambe le tipologie.
- Sono stati esclusi tutti i file non incentrati sulle slide didattiche come:
 - documenti dedicati al CRISP-DM basati sulle pagine del libro "Machine Learning Engineering" di Burkov.
 - appunti del docente ed esercizi preparatori all'esame

Il motivo di tale esclusione è spinto dalla semplificazione del corpus RAG poichè il libro, scritto in lingua inglese, avrebbe potuto causare difficoltà nell'embedding dei chunk mediante query e difficoltà di multilingua e traduzione; gli appunti del docente, a loro volta, rappresentavano materiale non ufficiale del corso sebbene risultassero di alta qualità didattica.

- I documenti sono stati rinominati e collocati in sottocartelle per modulo e argomento, garantendo uniformità e semplicità di gestione, alla fine della selezione il numero di documenti rimasto è diventato 15.

3.2 Pulizia dei dati

La fase di pulizia è stata finalizzata a rendere i documenti coerenti, leggibili e privi di rumore. Le principali operazioni hanno riguardato:

- Eliminazione di pagine vuote, copertine ridondanti e versioni duplicate.
- Accorpamento di slide ripetute con minime differenze ed eliminazione di quelle non significative.
- Verifica che i PDF fossero in formato testuale leggibile, evitando sezioni acquisite come immagini non processabili.
- Selezione delle immagini realmente utili al contenuto e conversione in descrizioni testuali di quelle poco chiare o malformate.
- Uniformazione della nomenclatura dei file e correzione di caratteri anomali, così da garantire un encoding coerente.
- Rimozione di elementi ridondanti (indici ripetuti, numerazioni incoerenti) potenzialmente fonte di disturbo.

L'intero processo è stato svolto a mano per ogni singolo documento. Al termine della pulizia si sono ottenuti documenti prevalentemente testuali, coerenti e pronti per la fase successiva. Sarà possibile ispezionare i pdf al percorso `Data_Extraction_&_Preparation/pdf_DataPrep/FIA`

3.3 Trasformazione dei dati e dataset finale

Completata la pulizia, i materiali sono stati trasformati per renderli adatti all'indicizzazione e al recupero tramite *Retrieval-Augmented Generation* (RAG). Il processo è stato condotto in più passaggi:

- **Chunking:** i PDF sono stati suddivisi in blocchi di testo (*chunk*) mediante l'ausilio di *ChatGPT-5 Reasoning*, rispettando un limite di circa 300 token con una sovrapposizione di 60 token tra blocchi consecutivi, così da mantenere continuità semantica. In questa fase i testi sono stati anche normalizzati per migliorarne la leggibilità. Per rendere trasparente il procedimento, di seguito è riportata la query utilizzata:

Esegui il chunking di questo PDF.

- Ogni chunk deve contenere al massimo 300 token.
- Usa un overlap di circa 60 token tra i chunk consecutivi.
- Non considerare il titolo delle slide (può essere identico in tutte).
- Basati solo sul contenuto testuale.
- Produci un JSON con struttura minimale:

```
{
  "doc_id": "...",
  "source": "<nome file PDF>",
  "chunks": [
    { "id": "c01", "text": "..." },
    { "id": "c02", "text": "..." },
    ...
  ]
}
```

```
]
}
```

- I testi devono essere puliti e coerenti, pronti per l'indicizzazione in FAISS.

- **Costruzione del corpus:** i chunk prodotti sono stati salvati in formato JSON per ogni documento e successivamente aggregati in un unico file tramite lo script `build_corpus`, ottenendo un corpus coerente e uniforme. Al percorso `Data_Extraction_&Preparation/corpus_rag_json/chunked_pdf` è possibile visionare i singoli json. Il corpus integrale è disponibile qui: `ALEC_Chatbot/src/corpus/data/corpus.json`
- **Embedding e indicizzazione:** il corpus JSON è stato trasformato in vettori semantici (*embedding*) attraverso un modello multilingua, quindi indicizzato con la libreria FAISS per consentire interrogazioni rapide ed efficienti.

Il risultato è un **dataset finale** costituito da chunk coerenti e arricchiti di meta-dati, organizzati in un database vettoriale FAISS. Questo dataset non rappresenta più un insieme statico di documenti, ma una base di conoscenza strutturata ed esplorabile tramite embedding semantici, su cui il sistema potrà effettuare interrogazioni e generare risposte pertinenti. Esso costituisce il riferimento ufficiale e validato su cui si fonda l'intero processo di supporto agli studenti.

4 Modeling

4.1 Introduzione e scelta del modello

Per la fase di modellazione è stato scelto il modello **Gemini Flash 2.5**, fornito da Google, per la sua capacità di generare testi coerenti e strutturati in tempi di risposta molto rapidi. La versione *Flash* del modello offre un buon compromesso tra qualità, velocità e costo computazionale, risultando particolarmente adatta ad applicazioni di tutoring interattivo in tempo reale. Poiché il progetto è stato sviluppato all'interno di un contesto accademico, si è scelto di utilizzare la **versione gratuita (free tier)** dell'API Gemini. Tale versione presenta vincoli di utilizzo in termini di:

- **RPM** (requests per minuto),
- **TPM** (token per minuto),
- **RPD** (requests per giorno).

Per gestire questi limiti è stato implementato un **rate limiter locale** (`rate_limiter.py`), che tiene traccia delle richieste effettuate e dei token consumati nel tempo. Il sistema mantiene finestre mobili per minuto e per giorno, bloccando automaticamente eventuali eccessi tramite eccezioni controllate:

```
if len(self.win_min) >= self.rpm:
    raise RuntimeError("Rate limit RPM locale raggiunto.")
```

Questa soluzione ha permesso di garantire la stabilità e l'affidabilità del sistema anche in presenza di più utenti contemporanei, rispettando i vincoli del piano gratuito dell'API.

4.2 Flusso di elaborazione del messaggio

L'interazione tra l'utente e il sistema segue una pipeline di elaborazione ben definita, che riflette direttamente i requisiti funzionali (RF1–RF6) e consente la produzione di risposte didattiche coerenti.

4.2.1 1. Ricezione del messaggio

Il server riceve ogni richiesta tramite l'endpoint `/message` e distingue tra:

- **messaggi standard**, che attivano la pipeline completa di risposta;
- **comandi speciali** (`/hint`, `/stats`, `/exit`), che seguono percorsi specifici.

4.2.2 2. Recupero del contesto

Lo script `retrieval.py` utilizza il modello di embedding **E5** e l'indice **FAISS** per individuare i *chunk* più pertinenti dai materiali del docente. Il contesto è poi ricomposto in un blocco leggibile tramite la funzione `build_context_block()`, che inserisce i nomi delle fonti ma esclude gli identificatori tecnici. Il numero di token è limitato dal parametro `MAX_CONTEXT_TOKENS`.

4.2.3 3. Valutazione (Judge)

Il messaggio utente e il contesto recuperato vengono passati al modello Gemini con il prompt `JUDGE_INSTRUCTIONS`. Il modello deve produrre un output rigorosamente in formato JSON, come mostrato di seguito:

```
{
  "wrongness": 0.0,
  "explanation": "...",
  "corrected": "...",
  "citations": [{"source": "...", "chunk_id": "..."}],
  "relevance": "pertinente" | "non_pertinente"
}
```

Questo output viene poi interpretato dal modulo `response_gate.py`, che decide l'azione successiva in base al valore di `wrongness`.

4.2.4 4. Decisione del Response Gate

Il *Response Gate* definisce la strategia di risposta secondo tre modalità principali:

- **Reinforce**: la risposta dell'utente è corretta; il sistema conferma e approfondisce brevemente.
- **Clarify**: la risposta è parzialmente corretta; il sistema fornisce chiarimenti mirati con scaffolding.
- **Correct**: la risposta è errata; il sistema produce una correzione completa utilizzando sempre la metodologia di scaffolding.

4.2.5 5. Generazione della risposta

Se l'azione scelta è *reinforce*, viene usato il prompt `ANSWER_INSTRUCTIONS`, che impone al modello di restituire solo HTML semplice, con eventuali fonti bibliografiche alla fine:

```
<p><b> Fonti:</b><br>...</p>
```

Nei casi di chiarimento o correzione viene invece utilizzato il prompt `SCAFFOLD_INSTRUCTIONS`, che chiede a Gemini di produrre un JSON con tre livelli di supporto progressivo:

```
{
  "level1": "<p>...</p>",
  "level2": "<p>...</p>",
  "level3": "<p>...</p>",
  "sources": ["..."]
}
```

I livelli vengono poi mostrati all'utente attraverso il comando `/hint`, che permette di visualizzare gradualmente l'aiuto senza alterare la cronologia della conversazione.

4.2.6 6. Gestione dei comandi speciali

- `/hint`: recupera i livelli successivi dello scaffolding e li visualizza.
- `/stats`: genera un riepilogo delle metriche di sessione tramite lo script `analytics.py`.
- `/exit`: chiude la conversazione, produce un report HTML finale con `report_generator.py` e disattiva l'interfaccia.

4.2.7 7. Output finale

La risposta prodotta (HTML o interpretata dal JSON) viene restituita al frontend (`homepage.html`), pronta per essere visualizzata. L'output rispetta sempre il vincolo di formattazione in HTML semplice, senza tag di struttura globale.

4.3 Progettazione dei prompt

I comportamenti del sistema ALEC sono modellati attraverso i prompt definiti in `prompts.py`, che rappresentano il cuore della fase di modellazione.

Prompt `SYSTEM_CORE` Definisce le regole generali di comunicazione del tutor: formato HTML, tono sintetico e uso controllato delle fonti.

Prompt `JUDGE_INSTRUCTIONS` Gestisce la valutazione logica della risposta dell'utente. Permette di distinguere risposte corrette, parziali o errate e di misurare il livello di *wrongness*.

Prompt `ANSWER_INSTRUCTIONS` Serve per la modalità di rinforzo positivo: il modello genera una risposta corretta e ben formattata in HTML semplice, citando le fonti consultate.

Prompt SCAFFOLD_INSTRUCTIONS Guida la costruzione dello *scaffolding* didattico a tre livelli, fornendo un supporto progressivo allo studente. I tre livelli vengono salvati e recuperati tramite il comando `/hint`.

Prompt REPORT_INSTRUCTIONS Usato per generare il report finale in HTML, diviso in due sezioni: **Docente** e **Studenti**. Riassume l'andamento della sessione, le fonti più usate e i suggerimenti personalizzati di studio.

4.4 Requisiti funzionali affrontati nella fase di modeling

Durante la fase di **Modeling** sono stati implementati e integrati tutti i requisiti funzionali (RF1–RF6), ognuno associato a specifici script e strategie di prompt. L'obiettivo principale di questa fase è stato quello di modellare il comportamento cognitivo del tutor ALEC, trasformando i requisiti progettuali in moduli software concreti e coerenti.

RF-1: Gestione della base di conoscenza e recupero delle fonti Il requisito RF1 è stato implementato attraverso lo script `retrieval.py`, che utilizza il modello di embedding **E5** e l'indice **FAISS** per individuare i *chunk* più pertinenti dal materiale docente. Una volta selezionate le porzioni di testo rilevanti, queste vengono trasformate in un blocco contestuale dal prompt definito nella funzione `build_context_block()` del file `prompts.py`. Questo prompt ha il compito di passare al modello Gemini i contenuti da considerare, mantenendo la tracciabilità delle **fonti** consultate. In questo modo, ogni risposta generata dal sistema può includere i riferimenti utilizzati, garantendo trasparenza e aderenza ai materiali forniti dal docente.

RF-2: Adattamento dinamico delle risposte e analisi del livello di errore Il requisito RF2 viene realizzato tramite lo script `response_gate.py`, che gestisce la logica decisionale del sistema sulla base del valore di **wrongness** prodotto dal modello nella fase di *judging*. La **wrongness** rappresenta un indice numerico compreso tra 0 e 1 che misura il grado di correttezza della risposta dell'utente: valori prossimi a 0 indicano risposte corrette, mentre valori prossimi a 1 segnalano errori significativi. In base a questo parametro, il sistema decide quale prompt utilizzare:

- **ANSWER_INSTRUCTIONS**: utilizzato quando la risposta è totalmente corretta o quasi corretta ($wrongness < 0.15$) per produrre un feedback positivo e sintetico;
- **SCAFFOLD_INSTRUCTIONS**: usato nei casi di errore o incertezza, per generare uno scaffolding didattico progressivo ($wrongness > 0.15$).

Questa distinzione consente al tutor di adattarsi dinamicamente alle necessità dello studente, fornendo rinforzi, chiarimenti o correzioni in base alla valutazione del modello.

RF-3: Scaffolding didattico e gestione locale delle risposte Per soddisfare il requisito RF3 è stata implementata una classe dedicata nello script `scaffolding.py`, che consente di mantenere in locale i livelli di aiuto generati

dal modello, evitando di inviare nuove query all'API Gemini. Questa soluzione aumenta significativamente l'efficienza complessiva del sistema, poiché riduce il numero di richieste al modello e quindi l'utilizzo di token. Lo *scaffolding* è costruito tramite il prompt `SCAFFOLD_INSTRUCTIONS`, che richiede la produzione di un JSON contenente tre livelli di supporto progressivo (`level1`, `level2`, `level3`) e l'elenco delle fonti consultate. L'utente può accedere ai livelli successivi di aiuto tramite il comando `/hint`, che recupera i dati dalla memoria locale senza riattivare la pipeline di elaborazione principale.

RF-4: Gestione multi-utente e multi-turno Il requisito RF4 riguarda la capacità del sistema di gestire più utenti e conversazioni multi-turno in parallelo. Questo aspetto è stato affrontato nella pipeline principale attraverso lo script `conversation_multi.py`, che differenzia i messaggi in base all'identificativo dell'utente e mantiene uno storico separato per ciascuna sessione. Durante la fase di modeling è stata definita la logica di assegnazione delle sessioni e la struttura dati che consente di recuperare il contesto pregresso per ogni interlocutore, permettendo così interazioni coerenti e personalizzate nel tempo.

RF-5: Analisi delle prestazioni e statistiche di apprendimento Il requisito RF5 è implementato tramite lo script `analytics.py`, che gestisce la raccolta e l'aggiornamento delle statistiche relative all'andamento dello studente e all'efficacia delle risposte del sistema. Le metriche vengono aggiornate automaticamente durante la fase di *judging* del modello, così da valutare ogni intervento in tempo reale. Attraverso il comando `/stats`, ogni utente può visualizzare un riepilogo delle proprie prestazioni, comprendente la pertinenza delle risposte, la qualità media delle interazioni e le fonti maggiormente consultate. Questo approccio consente allo studente di monitorare il proprio progresso e di orientare meglio lo studio.

RF-6: Chiusura della sessione e generazione del report finale Il requisito RF6 è stato pienamente modellato nella pipeline attraverso il comando `/exit`, che chiude la sessione e genera automaticamente un report finale in formato HTML. Lo script responsabile di questa funzione è `report_generator.py`, che salva il file nella cartella `src/report` e restituisce il link diretto al documento all'interno della chat. La generazione del report avviene mediante un prompt dedicato (`REPORT_INSTRUCTIONS`) inviato al modello Gemini, che produce un riepilogo strutturato in due sezioni principali:

- **Docente:** riassume l'andamento complessivo della sessione, le fonti più utilizzate e le principali difficoltà riscontrate;
- **Studenti:** sintetizza per ogni partecipante i punti di forza, le aree di miglioramento e suggerimenti di studio personalizzati.

Il report viene così reso disponibile sia localmente che direttamente nella conversazione, completando il ciclo di interazione didattica previsto dal sistema.

4.5 Scelta dei parametri di modellazione

Durante la progettazione e l'implementazione della fase di modeling sono stati selezionati alcuni parametri chiave per garantire un equilibrio tra efficienza,

qualità delle risposte e rispetto dei vincoli tecnici del piano gratuito dell'API Gemini. I principali parametri ottimizzati riguardano:

- il numero di documenti recuperati dal sistema di *retrieval* (`TOP_K`);
- il limite massimo di token per ogni contesto inviato al modello (`MAX_CONTEXT_TOKENS`);
- le soglie di errore (**wrongness**) che determinano le transizioni del *Response Gate*;

Tali parametri sono stati calibrati in modo empirico, verificando la qualità delle risposte e i tempi medi di elaborazione. I parametri sono ispezionabili nello script `variables.py`. Il risultato si presenta capace di mantenere una latenza contenuta e di fornire risposte coerenti, aderenti alle fonti e pienamente compatibili con le risorse disponibili nel contesto accademico.

5 Evaluation

5.1 Obiettivi della fase di valutazione

La fase di **Evaluation** ha l'obiettivo di verificare che il sistema ALEC, nella configurazione ottenuta durante il *Modeling*, soddisfi i requisiti funzionali e qualitativi definiti in precedenza. In particolare, vengono valutate le prestazioni del modello **Gemini Flash 2.5** e l'efficacia della pipeline nella produzione di risposte didatticamente coerenti, pertinenti e tempestive. La valutazione non si limita a misure di correttezza tecnica, ma tiene conto anche della qualità dell'interazione e dell'utilità percepita dallo studente nel contesto di peer tutoring.

5.2 Metodologia di valutazione

La validazione del sistema è stata condotta parallelamente alla fase di modeling testando ripetutamente domande e interazioni varie con il modello nel contesto multiturno. Nella cartella `src/evaluation` è possibile individuare due brevi chat tipo che sono state utilizzate per testare le metriche individuate nelle sezioni successive. Sono stati definiti diversi criteri di analisi:

- **Coerenza delle risposte:** verifica che le risposte fornite dal modello siano aderenti al contenuto dei materiali docente recuperati tramite il modulo di *retrieval*.
- **Correttezza logica e lessicale:** controllo della qualità del testo generato e della coerenza tra domanda, contesto e risposta prodotta.
- **Adattamento dinamico:** valutazione della capacità del sistema di distinguere correttamente i tre casi di *reinforce*, *clarify* e *correct* in base al valore di **wrongness**.
- **Efficienza e tempi di risposta:** analisi della latenza media end-to-end del sistema, considerando i vincoli del piano gratuito (*free tier*) dell'API Gemini.

- **Efficacia pedagogica:** misurazione qualitativa del supporto fornito tramite scaffolding e feedback, osservando la chiarezza, gradualità e pertinenza dei suggerimenti generati.

5.3 Valutazione delle funzionalità interattive

Le funzionalità aggiuntive di ALEC sono state anch'esse oggetto di valutazione:

- **Comando /hint:** ha mostrato un comportamento stabile, con recupero corretto dei livelli di scaffolding locali senza ulteriori chiamate al modello.
- **Comando /stats:** ha fornito statistiche aggiornate e coerenti grazie al modulo `analytics.py`, consentendo agli utenti di monitorare il proprio progresso.
- **Comando /exit:** ha generato correttamente i report finali in formato HTML, salvati nella cartella `src/report` e inviati in chat all'utente. I report risultano leggibili, coerenti e ben formattati.

5.4 Riscontro rispetto ai requisiti funzionali

I risultati complessivi confermano che tutti i requisiti funzionali (RF1–RF6) sono stati soddisfatti nella fase di modeling e validati durante la valutazione:

- **RF1:** il sistema recupera correttamente le fonti e le cita nelle risposte;
- **RF2:** il comportamento adattivo del tutor è coerente con la valutazione del modello;
- **RF3:** lo scaffolding progressivo rispetta i tre livelli previsti e riduce il numero di query al modello;
- **RF4:** il sistema gestisce correttamente più utenti e sessioni multi-turno;
- **RF5:** le statistiche di apprendimento sono raccolte e visualizzabili in tempo reale;
- **RF6:** la chiusura della sessione e la generazione del report finale avvengono in modo completo e stabile.

5.5 Valutazione dei requisiti non funzionali

Nel corso della fase di testing sono stati verificati i principali **requisiti non funzionali**, al fine di valutare la qualità complessiva del sistema e la sua rispondenza agli obiettivi progettuali. I risultati ottenuti possono essere così sintetizzati:

- **RNF-1 (Accuratezza)** — Il sistema ha raggiunto un livello di accuratezza complessivo prossimo al 100%, fornendo risposte corrette e coerenti con le fonti disponibili. Le valutazioni manuali e le metriche di wrongness osservata hanno confermato l'elevata affidabilità del modello nel contesto dei test di laboratorio.

- **RNF-2 (Prestazioni back-end)** — I tempi medi di risposta del tutor si sono mantenuti entro i limiti stabiliti, con una media di circa **36 secondi** per messaggio elaborato, rientrando nella soglia massima prevista di 40s. Il sistema risulta quindi performante rispetto alle risorse del piano free tier del modello utilizzato.
- **RNF-3 (Trasparenza)** — È stata costantemente garantita la presenza di citazioni alle fonti interne del materiale didattico per ogni correzione o chiarimento fornito dal tutor. Questo ha permesso di mantenere un livello adeguato di tracciabilità e trasparenza nelle risposte, favorendo la verificabilità dei contenuti.

5.6 Considerazioni finali sulla fase di valutazione

La fase di *Evaluation* ha dimostrato che il sistema ALEC soddisfa pienamente gli obiettivi progettuali di accuratezza, efficienza e valore didattico. Il modello Gemini Flash 2.5, integrato con la pipeline di script sviluppata, ha mostrato un comportamento affidabile e coerente anche all'interno dei limiti imposti dal piano gratuito. I risultati ottenuti confermano una discreta solidità della modellazione e costituiscono la base per il passaggio alla successiva fase di **Deployment**.

6 Deployment

6.1 Introduzione

La fase di **Deployment** ha avuto come obiettivo la realizzazione di un **MVP (Minimum Viable Prototype)** del sistema ALEC, concepito come una piattaforma di tutoraggio intelligente basata su un modello linguistico integrato con meccanismi di Retrieval-Augmented Generation (RAG). In questa fase non si è mirato alla produzione di un sistema pienamente distribuito o scalabile, ma alla verifica della **fattibilità tecnica e didattica** del progetto. Il prototipo è stato progettato e testato interamente in ambiente **locale**, utilizzando **FastAPI** come framework backend e una semplice interfaccia web per la chat. La scelta di un deployment locale è stata dettata sia da vincoli di tempo e risorse del tirocinio, sia dalla necessità di garantire controllo totale sul flusso dei dati e sul comportamento del modello durante la fase di valutazione.

6.2 Caratteristiche del prototipo

L'MVP implementa tutte le componenti fondamentali del sistema, già validate nella fase di *Modeling* e testate nella *Evaluation*. In particolare:

- la pipeline **Retrieval → Judge → Gate → Answer/Scaffolding** è pienamente operativa e integrata;
- il sistema supporta la gestione **multi-utente e multi-turno**, con sessioni distinte per ciascun interlocutore;
- sono attivi i comandi funzionali **/hint**, **/stats** e **/exit**, che consentono di accedere rispettivamente allo scaffolding, alle statistiche di sessione e al report finale;

- la generazione delle risposte è affidata al modello **Gemini Flash 2.5**, utilizzato tramite API nel piano *free tier*, con meccanismi locali di *rate limiting* per rispettare i vincoli di utilizzo.

Il deployment in locale ha consentito di verificare l'interazione completa tra i moduli, l'efficienza del recupero dei contenuti tramite FAISS e la qualità dei prompt nella generazione dei testi HTML. Tuttavia, non sono ancora presenti componenti di distribuzione o containerizzazione, poiché l'obiettivo primario era dimostrare la **coerenza del flusso e la correttezza funzionale** del sistema.

6.3 Limitazioni e miglioramenti futuri

Essendo un prototipo, ALEC presenta alcune limitazioni strutturali che costituiscono i principali punti di miglioramento per eventuali evoluzioni future:

- **Deployment cloud:** attualmente il sistema funziona in locale; la migrazione su un ambiente cloud (ad esempio Google Cloud Run o Azure App Service) permetterebbe maggiore accessibilità e monitoraggio.
- **Gestione persistente dei dati:** la cronologia delle conversazioni e le statistiche vengono mantenute solo durante la sessione; un database relazionale o documentale (come PostgreSQL o MongoDB) migliorerebbe la tracciabilità e la continuità dell'apprendimento.
- **Ottimizzazione delle prestazioni:** l'indice FAISS e le chiamate API potrebbero essere ottimizzati per tempi di risposta inferiori e gestione più efficiente dei token.
- **Espansione del dominio di conoscenza:** l'attuale corpus si basa sui materiali del corso di Fondamenti di Intelligenza Artificiale; in futuro potrebbe essere esteso ad altri moduli didattici o a un'intera piattaforma di peer learning.
- **Interfaccia utente:** l'interfaccia web è minimale e funzionale, ma può essere evoluta in una dashboard interattiva con grafici, progress tracker e feedback adattivo.

6.4 Valutazione del deployment

Nonostante la natura sperimentale, il deployment locale ha dimostrato la **fattibilità tecnica** e la **consistenza logica** dell'architettura ALEC. Tutte le fasi della pipeline risultano operative, e i risultati ottenuti nella fase di test confermano la validità dell'approccio RAG integrato con un modello linguistico moderno. Il sistema, pur in versione prototipale, è in grado di:

- fornire risposte pertinenti e coerenti ai materiali di riferimento;
- adattarsi al livello dello studente mediante scaffolding progressivo;
- monitorare e sintetizzare l'attività didattica tramite statistiche e report automatici.

6.5 Conclusione

La fase di **Deployment** conclude il ciclo di sviluppo del progetto ALEC, sancendo la trasformazione di un insieme di moduli sperimentali in un **prototipo funzionante e valutabile**. L'obiettivo principale era dimostrare la **fattibilità tecnica** e la **validità concettuale** dell'approccio proposto, più che ottenere un sistema completo e pronto alla produzione. In questo senso, il progetto ha raggiunto pienamente il proprio scopo: ha mostrato che un tutor intelligente, basato su tecniche di *Retrieval-Augmented Generation* e integrato con un modello linguistico moderno come **Gemini Flash 2.5**, può operare efficacemente come strumento di supporto all'apprendimento universitario.

Nonostante la natura prototipale e le limitazioni tecniche del deployment locale, ALEC si è dimostrato un sistema coerente, stabile e didatticamente utile. Le interazioni osservate nei test hanno evidenziato una buona capacità del modello di interpretare il contesto, di fornire risposte mirate e di adattare il livello di dettaglio in base alle difficoltà dello studente. Questo rappresenta un risultato significativo, poiché valida non solo l'architettura software ma anche il concetto pedagogico alla base del progetto: l'idea che un tutor virtuale, se opportunamente progettato, possa facilitare il *peer learning* e stimolare l'autonomia cognitiva. Il lavoro svolto costituisce dunque un primo passo verso lo sviluppo di un sistema più completo, che in futuro potrà essere esteso e distribuito su larga scala. Le possibili evoluzioni riguardano il miglioramento dell'interfaccia, la persistenza dei dati, l'ottimizzazione delle chiamate API e l'ampliamento del dominio conoscitivo. Tuttavia, l'elemento più importante emerso da questa sperimentazione è la **conferma della fattibilità del paradigma ALEC**: un ambiente di apprendimento collaborativo basato sull'integrazione di modelli linguistici e conoscenze strutturate, capace di adattarsi al ritmo dello studente e di restituire un feedback progressivo, trasparente e misurabile.

L'intero progetto non rappresenta soltanto un esercizio tecnico, ma un **proof of concept** del potenziale educativo dell'intelligenza artificiale applicata alla didattica. Il percorso intrapreso ha fornito solide basi per sviluppi futuri e ha dimostrato come anche un deployment locale, se ben progettato, possa offrire risultati significativi nel validare l'efficacia di un sistema complesso come questo.